

01_schema-alert-py

#python #api #fastapi #swagger #pydantic #backend #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ schemas/  
            └─ alert.py
```

El archivo `alert.py` se encuentra dentro de la carpeta de schemas del backend:

```
backend/app/schemas/
```

Este archivo define los schemas relacionados con las alertas del laboratorio SIEM MVP.

Su función es controlar cómo se devuelven las alertas desde la API y qué datos se permiten modificar cuando se actualiza una alerta.

En concreto, define:

```
AlertStatus  
AlertOut  
AlertUIOut  
AlertUpdate
```

Estos schemas no crean tablas directamente en PostgreSQL. Su función es validar datos y dar forma a las respuestas HTTP.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,320p' backend/app/schemas/alert.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,320p'
```

Indica que se impriman las líneas de la 1 a la 320.

```
backend/app/schemas/alert.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```

from __future__ import annotations

from datetime import datetime
from typing import Optional, Literal

from pydantic import BaseModel, Field

AlertStatus = Literal["open", "ack", "closed"]


class AlertOut(BaseModel):
    id: int
    rule_id: int
    event_id: int
    title: str

    group_key: Optional[str] = None
    status: AlertStatus

    created_at: datetime
    updated_at: datetime

    model_config = {"from_attributes": True}


class AlertUIOut(AlertOut):
    # Campos extra "listos para UI" (sin tocar BD)
    rule_name: str
    event_ts: datetime
    event_source: str
    event_severity: int
    event_message: str


class AlertUpdate(BaseModel):
    # Permitimos solo cambios de estado en el MVP
    status: AlertStatus = Field(...)

```

4 Función general del archivo

El archivo `schemas/alert.py` define la estructura de las respuestas y actualizaciones relacionadas con alertas.

Su objetivo principal es controlar tres cosas:

1. Qué estados puede tener una alerta.
2. Qué campos devuelve una alerta básica.
3. Qué campos devuelve una alerta enriquecida para interfaz.
4. Qué campos se pueden modificar al actualizar una alerta.

La relación general es:

```

Modelo SQLAlchemy Alert
    ↓
AlertOut
    ↓
respuesta básica de API

```

Y para la interfaz:

```

Alert + Rule + Event
    ↓
AlertUIOut

```

↓
respuesta enriquecida para frontend

Para actualizar una alerta:

PATCH /alerts/{alert_id}
↓
AlertUpdate
↓
cambio de status

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
from __future__ import annotations
```

Importación futura para anotaciones modernas.

```
from datetime import datetime
from typing import Optional, Literal
```

Importaciones estándar para fechas y tipos.

```
from pydantic import BaseModel, Field
```

Importaciones de Pydantic.

```
AlertStatus = Literal["open", "ack", "closed"]
```

Definición de los estados válidos de una alerta.

```
class AlertOut(BaseModel):
    ...
```

Schema de salida básica de alertas.

```
class AlertUIOut(AlertOut):
    ...
```

Schema de salida enriquecida para interfaz.

```
class AlertUpdate(BaseModel):
    ...
```

Schema para actualizar el estado de una alerta.

Visualmente:

```
alert.py
├─ Importaciones
├─ AlertStatus
├─ AlertOut
│   ├─ id
│   ├─ rule_id
│   ├─ event_id
│   ├─ title
│   ├─ group_key
│   ├─ status
│   ├─ created_at
│   └─ updated_at
```

```
|   └─ model_config
└─ AlertUIOut
|   └─ rule_name
|   └─ event_ts
|   └─ event_source
|   └─ event_severity
|   └─ event_message
└─ AlertUpdate
    └─ status
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Esta línea activa el comportamiento moderno de Python para las anotaciones de tipos.

Permite que las anotaciones se gestionen de forma más flexible.

En este archivo no hay una anotación especialmente compleja, pero mantiene coherencia con otros módulos del proyecto.

Importación de `datetime`

```
from datetime import datetime
```

Esta línea importa `datetime` desde el módulo estándar `datetime`.

Se utiliza para campos de fecha y hora:

```
created_at: datetime
updated_at: datetime
event_ts: datetime
```

Estos campos aparecen en las respuestas de la API y representan momentos temporales relacionados con la alerta o el evento asociado.

Importación de `Optional` y `Literal`

```
from typing import Optional, Literal
```

Esta línea importa dos tipos desde el módulo `typing`.

`Optional`

```
Optional
```

Indica que un valor puede ser de un tipo concreto o puede ser `None`.

En este archivo se usa aquí:

```
group_key: Optional[str] = None
```

Esto significa que `group_key` puede ser una cadena o puede no existir.

`Literal`

```
Literal
```

Permite limitar un valor a un conjunto concreto de opciones.

En este archivo se usa para definir:

```
AlertStatus = Literal["open", "ack", "closed"]
```

Esto significa que el estado de una alerta solo puede ser uno de esos tres valores.

Importación de BaseModel y Field

```
from pydantic import BaseModel, Field
```

Esta línea importa dos elementos de Pydantic.

BaseModel

`BaseModel` permite crear schemas de validación y serialización.

Las clases que heredan de `BaseModel` pueden:

- Validar datos de entrada.
- Definir respuestas de salida.
- Convertir objetos ORM en JSON.
- Aplicar restricciones de tipos.

En este archivo se usa en:

```
class AlertOut(BaseModel):
```

y:

```
class AlertUpdate(BaseModel):
```

Field

`Field` permite definir campos con validaciones o marcar campos como obligatorios.

En este archivo se usa aquí:

```
status: AlertStatus = Field(...)
```

Los puntos suspensivos `...` indican que el campo es obligatorio.

Definición de AlertStatus

```
AlertStatus = Literal["open", "ack", "closed"]
```

Esta línea define un alias de tipo llamado `AlertStatus`.

Desglose:

```
AlertStatus
```

Nombre del tipo.

```
=
```

Asignación.

```
Literal["open", "ack", "closed"]
```

Indica que solo se permiten tres valores:

```
open
ack
closed
```

Este alias se reutiliza en varios schemas:

```
status: AlertStatus
```

y:

```
status: AlertStatus = Field(...)
```

Esto evita repetir la lista de estados en varios sitios.

Significado de los estados

Los estados definidos representan un ciclo de vida básico tipo SOC:

```
open
```

Alerta abierta o nueva. Todavía está pendiente de revisión.

```
ack
```

Alerta reconocida. Un analista la ha visto o la tiene en seguimiento.

```
closed
```

Alerta cerrada. Se considera revisada, resuelta o descartada.

Flujo conceptual:

```
open → ack → closed
```

Definición de `AlertOut`

```
class AlertOut(BaseModel):
```

Esta línea define el schema `AlertOut`.

Este schema representa la salida básica de una alerta desde la API.

Se utiliza en endpoints como:

```
GET /alerts
GET /alerts/{alert_id}
PATCH /alerts/{alert_id}
```

`AlertOut` contiene los datos principales de la tabla `alerts`.

Campo `id`

```
id: int
```

Define el identificador único de la alerta.

Este valor procede de PostgreSQL.

No lo introduce manualmente el usuario.

Relación:

```
Alert.id → id de la alerta en la tabla alerts
```

Campo rule_id

```
rule_id: int
```

Define el identificador de la regla que generó la alerta.

Este campo conecta la alerta con la tabla rules .

Relación:

```
alerts.rule_id → rules.id
```

Esto permite saber qué regla provocó la detección.

Campo event_id

```
event_id: int
```

Define el identificador del evento que disparó la alerta.

Este campo conecta la alerta con la tabla events .

Relación:

```
alerts.event_id → events.id
```

Esto permite rastrear qué evento concreto generó la alerta.

Campo title

```
title: str
```

Define el título de la alerta.

En ingest.py , este título se genera así:

```
title=f"Rule matched: {rule.name}"
```

Por tanto, el título suele indicar qué regla ha coincidido.

Ejemplo:

```
Rule matched: Failed login auth
```

Separación visual

La línea en blanco entre title y group_key no afecta al funcionamiento.

Sirve para separar visualmente los identificadores principales de los campos de clasificación y estado.

Campo `group_key`

```
group_key: Optional[str] = None
```

Define la clave de agrupación de la alerta.

Desglose:

```
group_key
```

Nombre del campo.

```
Optional[str]
```

Puede ser una cadena o `None`.

```
-
```

Valor por defecto.

El `group_key` suele derivarse de:

```
meta.host
```

en el evento recibido.

Ejemplo:

```
{  
  "group_key": "server-01"  
}
```

Este campo es importante para:

- agrupar alertas
- aplicar anti-duplicado
- aplicar throttle
- aplicar threshold

Campo `status`

```
status: AlertStatus
```

Define el estado de la alerta.

El tipo `AlertStatus` limita los valores válidos a:

```
open  
ack  
closed
```

Esto evita estados no controlados.

Ejemplo válido:

```
{  
  "status": "open"  
}
```


Ejemplo inválido:

```
{
  "status": "resolved"
}
```

`resolved` no está permitido por el schema.

Campo `created_at`

```
created_at: datetime
```

Define la fecha y hora de creación de la alerta.

Este valor procede del modelo `Alert`, donde se configura con:

```
server_default=func.now()
```

Es decir, PostgreSQL asigna automáticamente la fecha al crear la alerta.

Campo `updated_at`

```
updated_at: datetime
```

Define la fecha y hora de última actualización de la alerta.

Este campo es especialmente importante cuando se cambia el estado de una alerta.

Por ejemplo:

```
open → ack
ack → closed
```

Al actualizarse la alerta, `updated_at` permite saber cuándo se produjo el último cambio.

Configuración `model_config`

```
model_config = {"from_attributes": True}
```

Esta línea configura Pydantic para poder construir el schema desde atributos de objetos.

Es importante porque los endpoints devuelven objetos SQLAlchemy `Alert`.

Ejemplo:

```
return alert
```

`alert` no es un diccionario puro. Es una instancia ORM.

Gracias a:

```
model_config = {"from_attributes": True}
```

Pydantic puede leer:

```
alert.id
alert.rule_id
alert.event_id
alert.title
alert.group_key
```

```
alert.status
alert.created_at
alert.updated_at
```

y convertirlo en JSON.

Sin esta configuración, Pydantic podría no serializar correctamente objetos ORM.

Definición de `AlertUIOut`

```
class AlertUIOut(AlertOut):
```

Esta línea define el schema `AlertUIOut`.

Desglose:

```
class
```

Palabra clave para definir una clase.

```
AlertUIOut
```

Nombre del schema.

```
(AlertOut)
```

Indica que hereda de `AlertOut`.

Esto significa que `AlertUIOut` incluye todos los campos de `AlertOut` y añade otros campos adicionales.

La relación es:

```
AlertOut
  ↓
AlertUIOut
```

Por tanto, `AlertUIOut` tiene:

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

más los campos extra definidos debajo.

Comentario sobre campos de UI

```
# Campos extra "listos para UI" (sin tocar BD)
```

Este comentario explica la finalidad de `AlertUIOut`.

Los campos adicionales no pertenecen directamente a la tabla `alerts`.

Se calculan mediante consultas con join a las tablas:

```
rules
events
```

Esto permite devolver a la interfaz datos ya preparados.

No se modifica la base de datos. Solo se construye una respuesta enriquecida.

Campo `rule_name`

```
rule_name: str
```

Define el nombre de la regla asociada a la alerta.

Este campo procede de:

```
Rule.name
```

En `alerts.py`, se obtiene mediante:

```
Rule.name.label("rule_name")
```

Esto permite que la UI muestre el nombre de la regla sin tener que hacer otra llamada a `/rules`.

Campo `event_ts`

```
event_ts: datetime
```

Define el timestamp del evento asociado a la alerta.

Procede de:

```
Event.ts
```

En `alerts.py`, se obtiene mediante:

```
Event.ts.label("event_ts")
```

Esto permite saber cuándo ocurrió el evento que disparó la alerta.

Campo `event_source`

```
event_source: str
```

Define el origen del evento asociado.

Procede de:

```
Event.source
```

Ejemplo:

```
auth  
firewall  
linux
```

Este campo ayuda a clasificar rápidamente de dónde viene la alerta.

Campo `event_severity`

```
event_severity: int
```

Define la severidad del evento asociado.

Procede de:

```
Event.severity
```

Este campo permite mostrar en la UI la gravedad del evento que originó la alerta.

Campo `event_message`

```
event_message: str
```

Define el mensaje del evento asociado.

Procede de:

```
Event.message
```

Este campo permite ver el contexto principal del evento sin abrir otra vista o hacer otra consulta.

Resultado de `AlertUIOut`

`AlertUIOut` representa una alerta enriquecida.

Incluye los datos básicos de la alerta:

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

Y añada contexto:

```
rule_name
event_ts
event_source
event_severity
event_message
```

Esto permite que el frontend reciba una estructura más útil:

```
{
  "id": 1,
  "rule_id": 2,
  "event_id": 15,
  "title": "Rule matched: Failed login auth",
  "group_key": "server-01",
  "status": "open",
  "created_at": "2026-01-15T12:00:00",
  "updated_at": "2026-01-15T12:00:00",
  "rule_name": "Failed login auth",
  "event_ts": "2026-01-15T11:59:58",
  "event_source": "auth",
  "event_severity": 4,
  "event_message": "Failed login attempt for user admin"
}
```

Definición de `AlertUpdate`

```
class AlertUpdate(BaseModel):
```

Esta línea define el schema utilizado para actualizar una alerta.

Se usa en:

```
PATCH /alerts/{alert_id}
```

Este schema no permite modificar todos los campos de una alerta.

Solo permite modificar el estado.

Comentario sobre actualización

```
# Permitimos solo cambios de estado en el MVP
```

Este comentario explica una decisión de diseño.

En este MVP, el usuario no puede modificar:

```
rule_id
event_id
title
group_key
created_at
updated_at
```

Solo puede cambiar:

```
status
```

Esto es coherente con una gestión básica de alertas.

El evento y la regla que generaron la alerta no deberían modificarse desde la actualización de estado.

Campo `status` en `AlertUpdate`

```
status: AlertStatus = Field(...)
```

Define el estado nuevo de la alerta.

Desglose:

```
status
```

Nombre del campo.

```
AlertStatus
```

Tipo permitido. Solo acepta:

```
open
ack
closed
```

```
Field(...)
```

Indica que el campo es obligatorio.

El cliente debe enviar un JSON como:

```
{
  "status": "ack"
}
```

o:

```
{
  "status": "closed"
}
```

Si envía otro valor, FastAPI devolverá error de validación.

Resultado final del archivo

Después de cargar este archivo, quedan disponibles:

```
AlertStatus
AlertOut
AlertUIOut
AlertUpdate
```

Resumen:

```
AlertStatus
└─ open / ack / closed

AlertOut
├─ id
├─ rule_id
├─ event_id
├─ title
├─ group_key
├─ status
├─ created_at
└─ updated_at

AlertUIOut
├─ todos los campos de AlertOut
├─ rule_name
├─ event_ts
├─ event_source
├─ event_severity
└─ event_message

AlertUpdate
└─ status
```

Relación con el flujo técnico del laboratorio

Este archivo participa en la fase final del flujo SIEM: la consulta y gestión de alertas.

Flujo de generación:

```
POST /ingest
  ↓
se crea Event
  ↓
se evalúa Rule
```

↓
se crea Alert

Flujo de consulta básica:

GET /alerts
↓
consulta Alert
↓
devuelve AlertOut

Flujo de consulta para frontend:

GET /alerts/ui
↓
consulta Alert + Rule + Event
↓
devuelve AlertUIOut

Flujo de actualización:

PATCH /alerts/{alert_id}
↓
recibe AlertUpdate
↓
modifica Alert.status
↓
devuelve AlertOut

8 Errores típicos o puntos importantes

AlertStatus limita los estados válidos

Solo se permiten:

open
ack
closed

Esto evita inconsistencias en el ciclo de vida de las alertas.

AlertUIOut hereda de AlertOut

No repite los campos básicos.

Los hereda y añade campos preparados para UI.

Esto evita duplicar estructuras.

AlertUIOut no representa una tabla nueva

Aunque tenga más campos, no significa que exista una tabla diferente.

AlertUIOut es una respuesta enriquecida.

Los campos extra salen de joins con Rule y Event .

AlertUpdate solo modifica estado

Esto protege la integridad de la alerta.

El usuario no puede cambiar desde este schema el evento original, la regla asociada o el título generado.

Field(...) hace obligatorio el estado

En `AlertUpdate`, el campo `status` debe enviarse siempre.

Esto evita peticiones vacías como:

```
{}
```

model_config permite responder con objetos ORM

`AlertOut` necesita:

```
model_config = {"from_attributes": True}
```

porque los endpoints devuelven objetos SQLAlchemy.

9 Comandos útiles relacionados

Consultar alertas básicas:

```
curl http://localhost:8000/alerts
```

Consultar alertas enriquecidas para UI:

```
curl http://localhost:8000/alerts/ui
```

Consultar una alerta concreta:

```
curl http://localhost:8000/alerts/1
```

Consultar una alerta concreta enriquecida:

```
curl http://localhost:8000/alerts/1/ui
```

Actualizar una alerta a `ack`:

```
curl -X PATCH http://localhost:8000/alerts/1 \
  -H "Content-Type: application/json" \
  -d '{
    "status": "ack"
  }'
```

Actualizar una alerta a `closed`:

```
curl -X PATCH http://localhost:8000/alerts/1 \
  -H "Content-Type: application/json" \
  -d '{
    "status": "closed"
  }'
```

Probar error por estado inválido:

```
curl -X PATCH http://localhost:8000/alerts/1 \
  -H "Content-Type: application/json" \
  -d '{
    "status": "resolved"
  }'
```

Probar importación del schema:


```
docker exec -it siem-api python -c "from app.schemas.alert import AlertOut, AlertUIOut, AlertUpdate, AlertStatus; print(AlertOut, AlertUIOut, AlertUpdate, AlertStatus)"
```

Comprobar Swagger:

```
http://localhost:8000/docs
```